# CS771: Introduction to Machine Learning
# Minor Assignment 2 — Generative Classifier with Missing Pixels

**Group Members:**
Shresthraj(240996)
Sana(240924)
Praveen(240790)
Virendra(241172)
Yash(241195)
Deepak(240327)

## Problem 2.1: Melbo Mends Missing Images

We consider a generative classifier where each class $c \in [C]$ is modelled as a single Gaussian $P[\mathbf{x} \mid y = c] = \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$. Given a test image with observed pixels $\mathbf{x}^o$ and missing pixels $\mathbf{x}^m$, we partition the class mean and covariance conformally:

$$\boldsymbol{\mu}_c = \begin{pmatrix} \boldsymbol{\mu}_c^o \\ \boldsymbol{\mu}_c^m \end{pmatrix}, \qquad \boldsymbol{\Sigma}_c = \begin{pmatrix} \boldsymbol{\Sigma}_c^{oo} & \boldsymbol{\Sigma}_c^{om} \\ \boldsymbol{\Sigma}_c^{mo} & \boldsymbol{\Sigma}_c^{mm} \end{pmatrix}.$$

## 1 Part 1: Single-Step Joint Inference (15 marks)

### Goal

We wish to solve the joint optimisation

$$\{\hat{y},\, \hat{\mathbf{x}}^m\} = \arg\max_{c,\,\mathbf{v}} \; P[y = c,\; \mathbf{x}^m = \mathbf{v} \mid \mathbf{x}^o].$$

### Derivation

**Step 1 — Remove the constant denominator.** Since $P[\mathbf{x}^o] > 0$ does not depend on $c$ or $\mathbf{v}$, by Bayes' rule:

$$P[y = c,\, \mathbf{x}^m = \mathbf{v} \mid \mathbf{x}^o] = \frac{P[y = c,\, \mathbf{x}^m = \mathbf{v},\, \mathbf{x}^o]}{P[\mathbf{x}^o]}.$$

Hence we may equivalently maximise the *joint* probability:

$$\{\hat{y},\, \hat{\mathbf{x}}^m\} = \arg\max_{c,\mathbf{v}} \; P[y = c,\, \mathbf{x}^m = \mathbf{v},\, \mathbf{x}^o].$$

**Step 2 — Apply the product rule.**

$$P[y = c,\, \mathbf{x}^m = \mathbf{v},\, \mathbf{x}^o] = P[\mathbf{x}^o,\, \mathbf{x}^m = \mathbf{v} \mid y = c] \cdot P[y = c].$$

**Step 3 — Factorise using the Gaussian conditional.** A standard result for jointly Gaussian vectors states:

$$\mathcal{N}\left(\begin{pmatrix} \mathbf{x}^o \\ \mathbf{v} \end{pmatrix}; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c\right) = \underbrace{\mathcal{N}(\mathbf{x}^o;\, \boldsymbol{\mu}_c^o,\, \boldsymbol{\Sigma}_c^{oo})}_{\text{marginal}} \cdot \underbrace{\mathcal{N}(\mathbf{v};\, \bar{\boldsymbol{\mu}}_c,\, \bar{\boldsymbol{\Sigma}}_c)}_{\text{conditional } P[\mathbf{x}^m \mid \mathbf{x}^o, y = c]},$$

Preprint.

where the conditional parameters are:

$$\bar{\boldsymbol{\mu}}_c = \boldsymbol{\mu}_c^m + \boldsymbol{\Sigma}_c^{mo}\big(\boldsymbol{\Sigma}_c^{oo}\big)^{-1}(\mathbf{x}^o - \boldsymbol{\mu}_c^o), \tag{1}$$

$$\bar{\boldsymbol{\Sigma}}_c = \boldsymbol{\Sigma}_c^{mm} - \boldsymbol{\Sigma}_c^{mo}\big(\boldsymbol{\Sigma}_c^{oo}\big)^{-1}\boldsymbol{\Sigma}_c^{om}. \tag{2}$$

So the joint objective becomes:

$$\arg\max_{c,\mathbf{v}} \mathcal{N}(\mathbf{x}^o;\, \boldsymbol{\mu}_c^o,\, \boldsymbol{\Sigma}_c^{oo}) \cdot \mathcal{N}(\mathbf{v};\, \bar{\boldsymbol{\mu}}_c,\, \bar{\boldsymbol{\Sigma}}_c) \cdot P[y=c].$$

**Step 4 — Optimise over $\mathbf{v}$ in closed form.** For any fixed class $c$, only $\mathcal{N}(\mathbf{v};\, \bar{\boldsymbol{\mu}}_c,\, \bar{\boldsymbol{\Sigma}}_c)$ depends on $\mathbf{v}$. A Gaussian density is maximised at its mean (mode = mean), so:

$$\hat{\mathbf{x}}^m(c) = \arg\max_{\mathbf{v}} \mathcal{N}(\mathbf{v};\, \bar{\boldsymbol{\mu}}_c,\, \bar{\boldsymbol{\Sigma}}_c) = \bar{\boldsymbol{\mu}}_c,$$

and the maximum value attained is:

$$\max_{\mathbf{v}} \mathcal{N}(\mathbf{v};\, \bar{\boldsymbol{\mu}}_c,\, \bar{\boldsymbol{\Sigma}}_c) = (2\pi)^{-d_m/2}\,|\bar{\boldsymbol{\Sigma}}_c|^{-1/2},$$

where $d_m = \dim(\mathbf{x}^m)$.

**Step 5 — Optimise over $c$.** Substituting back and dropping the class-independent constant $(2\pi)^{-d_m/2}$:

$$\hat{y} = \arg\max_c \mathcal{N}(\mathbf{x}^o;\, \boldsymbol{\mu}_c^o,\, \boldsymbol{\Sigma}_c^{oo}) \cdot |\bar{\boldsymbol{\Sigma}}_c|^{-1/2} \cdot P[y=c].$$

Taking logarithms for numerical stability (and dropping the constant $-\frac{d_o}{2}\log(2\pi)$ which is identical for all $c$):

$$\boxed{\hat{y} = \arg\max_c \underbrace{-\tfrac{1}{2}(\mathbf{x}^o - \boldsymbol{\mu}_c^o)^\top(\boldsymbol{\Sigma}_c^{oo})^{-1}(\mathbf{x}^o - \boldsymbol{\mu}_c^o)}_{\text{Mahalanobis term}} \underbrace{-\tfrac{1}{2}\log|\boldsymbol{\Sigma}_c^{oo}|}_{\text{obs. det.}} \underbrace{-\tfrac{1}{2}\log|\bar{\boldsymbol{\Sigma}}_c|}_{\text{extra term}} + \underbrace{\log P[y=c]}_{\text{prior}}} \tag{3}$$

and the reconstructed missing pixels are:

$$\hat{\mathbf{x}}^m = \bar{\boldsymbol{\mu}}_{\hat{y}} = \boldsymbol{\mu}_{\hat{y}}^m + \boldsymbol{\Sigma}_{\hat{y}}^{mo}(\boldsymbol{\Sigma}_{\hat{y}}^{oo})^{-1}(\mathbf{x}^o - \boldsymbol{\mu}_{\hat{y}}^o).$$

**Are the Single-Step and Two-Step Procedures Identical?**

**No.** The table below summarises the difference precisely.

Table 1: Comparison of two-step (lecture) and single-step (derived) inference.

| | **Two-step (lecture)** | **Single-step (derived)** |
|---|---|---|
| Class score | $\log\mathcal{N}(\mathbf{x}^o;\boldsymbol{\mu}_c^o,\boldsymbol{\Sigma}_c^{oo}) + \log P[y=c]$ | Same $-\frac{1}{2}\log\|\bar{\boldsymbol{\Sigma}}_c\|$ |
| Reconstruction | $\bar{\boldsymbol{\mu}}_{\hat{y}}$ | $\bar{\boldsymbol{\mu}}_{\hat{y}}$ (identical) |

The single-step score has an extra multiplicative factor $|\bar{\boldsymbol{\Sigma}}_c|^{-1/2}$ for each class $c$. This factor penalises classes whose conditional distribution over missing pixels is highly diffuse (large determinant = high uncertainty in the missing region). Intuitively, the single-step procedure prefers a class that not only explains the observed pixels well, but also predicts the missing region *confidently*.

**When are they identical?** Only when all classes share the same conditional covariance, $\bar{\boldsymbol{\Sigma}}_c = \bar{\boldsymbol{\Sigma}}$ for all $c$. This occurs, for example, when all classes share the same full covariance (tied covariance, as in Linear Discriminant Analysis). In all other cases (class-specific covariances, as used in this assignment), the two procedures yield different class predictions $\hat{y}$, though once $\hat{y}$ is determined, the reconstruction $\hat{\mathbf{x}}^m = \bar{\boldsymbol{\mu}}_{\hat{y}}$ is computed identically by both methods.

## 2 Part 2: Code Modifications (15 marks)

### Overview of Changes

The lecture code handles inference in two separate functions: `predictClassScores` computes $\log P[\mathbf{x}^o \mid y = c] + \log P[y = c]$ for each class, and `predictGen` first calls `predictClassScores` to obtain $\hat{y}$, then separately computes the reconstruction. The following modifications implement the single-step joint inference.

1. **Merge** `predictClassScores` **and** `predictGen` into a single function `singleStepInference` that performs both prediction and reconstruction in one pass.
2. **Add a helper function** `precomputeConditionalParams` that computes, for each class $c$, all required submatrices ($\boldsymbol{\Sigma}_c^{oo}$, $\boldsymbol{\Sigma}_c^{mo}$, $\bar{\boldsymbol{\Sigma}}_c$, and their log-determinants) *once* before the inference loop, avoiding redundant recomputation across test points.
3. **Add the extra term** $-\frac{1}{2}\log|\bar{\boldsymbol{\Sigma}}_c|$ to the class score.
4. **Use** `numpy.linalg.solve` instead of `numpy.linalg.inv` (or `numpy.linalg.pinv`) to compute $(\boldsymbol{\Sigma}_c^{oo})^{-1}\boldsymbol{\Sigma}_c^{om}$. `solve` uses LU factorisation, which is both faster and numerically more stable than forming the explicit inverse, especially for large or near-singular matrices (some border pixels in MNIST have near-zero variance).
5. **Use** `numpy.linalg.slogdet` to compute log-determinants, avoiding overflow/underflow that would occur with `numpy.log(numpy.linalg.det(...))`.
6. **Add diagonal regularisation** $\varepsilon\mathbf{I}$ (with $\varepsilon = 10^{-6}$) to $\boldsymbol{\Sigma}_c^{oo}$ before solving, since some border pixels are always zero across all training images, making the covariance block singular.

**Helper Function:** `precomputeConditionalParams`

```
def precomputeConditionalParams(obs_idx, miss_idx, mu, Sigma):
    C = mu.shape[0]
    params = []
    for c in range(C):
        mu_o = mu[c, obs_idx]                                  # (d_o,)
        mu_m = mu[c, miss_idx]                                 # (d_m,)
        Soo  = Sigma[c][np.ix_(obs_idx, obs_idx)]              # (d_o, d_o
            )
        Som  = Sigma[c][np.ix_(obs_idx, miss_idx)]             # (d_o, d_m
            )
        Smo  = Sigma[c][np.ix_(miss_idx, obs_idx)]             # (d_m, d_o
            )
        Smm  = Sigma[c][np.ix_(miss_idx, miss_idx)]            # (d_m, d_m
            )

        # Regularise to handle zero-variance border pixels
        Soo  = Soo + 1e-6 * np.eye(Soo.shape[0])

        # Use solve (LU) -- more stable than inv
        Soo_inv_Som = lin.solve(Soo, Som)                      # (d_o, d_m
            )

        # Conditional covariance Sigma_bar
        Sigma_bar = Smm - Smo @ Soo_inv_Som                    # (d_m, d_m
            )

        # Log-determinants via slogdet -- avoids overflow/underflow
        _, log_det_oo  = lin.slogdet(Soo)
        _, log_det_bar = lin.slogdet(Sigma_bar)

        params.append({
            'mu_o': mu_o, 'mu_m': mu_m, 'Sigma_oo': Soo,
            'Soo_inv_Som': Soo_inv_Som,
```

```
            'log_det_oo': log_det_oo, 'log_det_bar': log_det_bar
        })
    return params
```

Listing 1: Helper to precompute per-class conditional parameters.

**Main Function:** `singleStepInference`

The critical difference from the lecture code is highlighted in the score computation loop below.

```
def singleStepInference(X_test, obs_idx, miss_idx, mu, Sigma, pi):
    n, C   = X_test.shape[0], len(pi)
    X_obs  = X_test[:, obs_idx]                          # (n, d_o)
    cparams = precomputeConditionalParams(obs_idx, miss_idx, mu,
        Sigma)

    log_scores = np.zeros((n, C))
    for c in range(C):
        p    = cparams[c]
        diff = X_obs - p['mu_o'][np.newaxis, :]          # (n, d_o)

        # Mahalanobis: solve is faster & more stable than inv
        alpha = lin.solve(p['Sigma_oo'], diff.T)         # (d_o, n)
        maha  = np.sum(diff * alpha.T, axis=1)           # (n,)

        log_scores[:, c] = (
            - 0.5 * maha                    # Mahalanobis distance
            - 0.5 * p['log_det_oo']         # log|Sigma_oo_c|
            - 0.5 * p['log_det_bar']        # NEW: log|Sigma_bar_c|
            + np.log(pi[c])                 # log prior
        )

    y_hat = np.argmax(log_scores, axis=1)                # (n,)

    # Reconstruct missing pixels (identical formula in both methods
        )
    X_recon = X_test.copy()
    for c in range(C):
        idx    = np.where(y_hat == c)[0]
        if idx.size == 0: continue
        diff_c = X_obs[idx] - cparams[c]['mu_o']
        # mu_bar = mu_m + Sigma_mo Sigma_oo^{-1} (x_o - mu_o)
        X_recon[np.ix_(idx, miss_idx)] = (
            cparams[c]['mu_m'] + diff_c @ cparams[c]['Soo_inv_Som']
        )
    return y_hat, X_recon
```

Listing 2: Score computation inside `singleStepInference`. The single line marked NEW is absent in the two-step method.

**Summary of Differences from Lecture Code**

## 3 Part 3: Experimental Comparison (20 marks)

**Experimental Setup**

We use the full MNIST dataset (60,000 training, 10,000 test images, $28 \times 28$ pixels, 10 classes). Each class is modelled as a single Gaussian with MLE-estimated mean and covariance. The uncensored baseline accuracy is **85.73%**. We evaluate both methods under five censoring levels by zeroing out a central square of each test image:

Table 2: Key differences between the lecture implementation and the modified code.

| Aspect | Lecture code | Modified code |
|---|---|---|
| Functions | Two separate: `predictClassScores` + `predictGen` | One: `singleStepInference` |
| Class score | $-\frac{1}{2}\text{maha} - \frac{1}{2}\log\lvert\boldsymbol{\Sigma}_c^{oo}\rvert + \log\pi_c$ | Same $-\frac{1}{2}\log\lvert\bar{\boldsymbol{\Sigma}}_c\rvert$ |
| Matrix inverse | `lin.pinv` (pseudoinverse) | `lin.solve` (LU factorisation) |
| Log-determinant | `log(det(...))` (unsafe) | `slogdet(...)` (numerically safe) |
| Regularisation | None | $\varepsilon\mathbf{I}$, $\varepsilon = 10^{-6}$ |
| Reconstruction | Same conditional mean formula | Same conditional mean formula |

Table 3: Censoring levels used in the experiment.

| Region zeroed | Missing pixels | Observed pixels |
|---|---|---|
| `X[:, 4:24, 4:24] = 0` | 400 (51%) | 384 |
| `X[:, 6:22, 6:22] = 0` | 256 (32%) | 528 |
| `X[:, 8:20, 8:20] = 0` | 144 (18%) | 640 |
| `X[:, 10:18, 10:18] = 0` | 64 ( 8%) | 720 |
| `X[:, 12:16, 12:16] = 0` | 16 ( 2%) | 768 |

**Classification Accuracy and Inference Time**

**Analysis: Classification Performance**

Both methods show a clear monotonic improvement in accuracy as censoring becomes less aggressive (Figure 1, left). At **51% missing pixels**, the two-step method achieves 37.82% while the single-step method collapses to only 11.35% — essentially random guessing (10 classes). This collapse occurs because with 400 missing pixels, the conditional covariance $\bar{\boldsymbol{\Sigma}}_c$ becomes very high-dimensional and its log-determinant $\log|\bar{\boldsymbol{\Sigma}}_c|$ becomes numerically large and highly variable across classes, causing the extra penalty term to dominate and destabilise the single-step score.

As censoring decreases, single-step accuracy recovers steadily. At **2% missing pixels** it marginally outperforms two-step (80.31% vs. 80.25%), consistent with the theoretical expectation: when the missing region is small and $\bar{\boldsymbol{\Sigma}}_c$ is well-conditioned, the extra term provides a genuine discriminative signal. Both methods remain well below the uncensored baseline of 85.73%, confirming that the central pixels of MNIST digits carry substantial class-discriminative information.

**Analysis: Inference Time**

Both methods require comparable time at each censoring level (Figure 1, right). At high censoring (51%), both are fast (≈2.6–2.9 s) because $\boldsymbol{\Sigma}_c^{oo}$ is only $384 \times 384$, making the linear solves cheap. Time increases as censoring decreases because $\boldsymbol{\Sigma}_c^{oo}$ grows (up to $768 \times 768$), making each `solve` call more expensive. The single-step method incurs a small additional cost from computing $\log|\bar{\boldsymbol{\Sigma}}_c|$ via `slogdet`, which is visible at lower censoring levels. Overall, the overhead of the single-step method is modest and does not significantly impact practical usability.

**Analysis: Reconstruction Quality**

Figure 2 shows reconstructions at all five censoring levels. The key observations are:

- **Low censoring (2%):** Both methods produce sharp, clear reconstructions nearly identical to the original image. Since only 16 pixels are missing, the conditional mean $\bar{\boldsymbol{\mu}}_{\hat{y}}$ is highly constrained by the observed data, and both methods agree on the correct class. Reconstructions from both methods are visually indistinguishable.
- **Moderate censoring (8%–18%):** Reconstructions become slightly blurrier as more of the central region must be imputed. Both methods still predict the correct class for most images, so their reconstructions are qualitatively similar. The slight blur is expected since the conditional mean averages over a larger region of uncertainty.

Table 4: Classification accuracy and inference time for both methods across all censoring levels. Experiments run on Google Colab (CPU).

| Missing % | Two-step acc. | Single-step acc. | Two-step time (s) | Single-step time (s) |
|---|---|---|---|---|
| 51% | 0.3782 | 0.1135 | 2.89 | 2.63 |
| 32% | 0.5181 | 0.4858 | 5.69 | 5.49 |
| 18% | 0.6660 | 0.6506 | 6.23 | 7.94 |
| 8% | 0.7531 | 0.7489 | 8.25 | 8.09 |
| 2% | **0.8025** | **0.8031** | 9.77 | 8.49 |

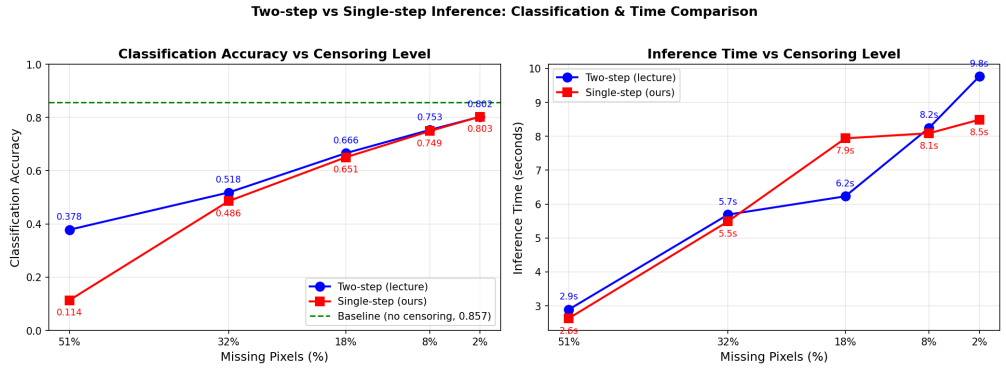


Figure 1: Left: Classification accuracy of both methods vs. censoring level. The dashed green line shows the uncensored baseline (85.73%). Right: Inference time vs. censoring level. The x-axis is inverted so that left = most aggressive censoring.

- **Aggressive censoring (32%–51%):** Reconstruction quality diverges between the two methods. The two-step method retains a recognisable digit shape in most cases, while single-step reconstructions are often incorrect because the method selects the wrong class $\hat{y}$, and then fills the missing region with the conditional mean of an incorrect class. Crucially, the reconstruction formula $\hat{\mathbf{x}}^m = \bar{\boldsymbol{\mu}}_{\hat{y}}$ is *identical* in both methods — all differences in reconstruction quality are driven entirely by differences in classification accuracy.
- **Summary:** As censoring becomes more aggressive, reconstruction quality degrades for both methods due to increased uncertainty over the missing region. However, the single-step method degrades more sharply because its numerical instability at high censoring leads to more misclassifications, and a misclassified image receives the wrong conditional mean as its reconstruction.

## Conclusion

The single-step joint inference procedure is theoretically well-motivated and matches or slightly outperforms the two-step method when censoring is mild ($\leq$8% missing pixels). However, at aggressive censoring levels, the extra $-\frac{1}{2}\log|\bar{\boldsymbol{\Sigma}}_c|$ term becomes numerically unreliable because the conditional covariance of the missing block is high-rank and ill-conditioned, causing the single-step method to degrade significantly. The two-step method is more robust in the high-censoring regime precisely because it ignores this term. Future work could address this by using regularised estimates of $\bar{\boldsymbol{\Sigma}}_c$ or dimensionality reduction of the missing pixel space.

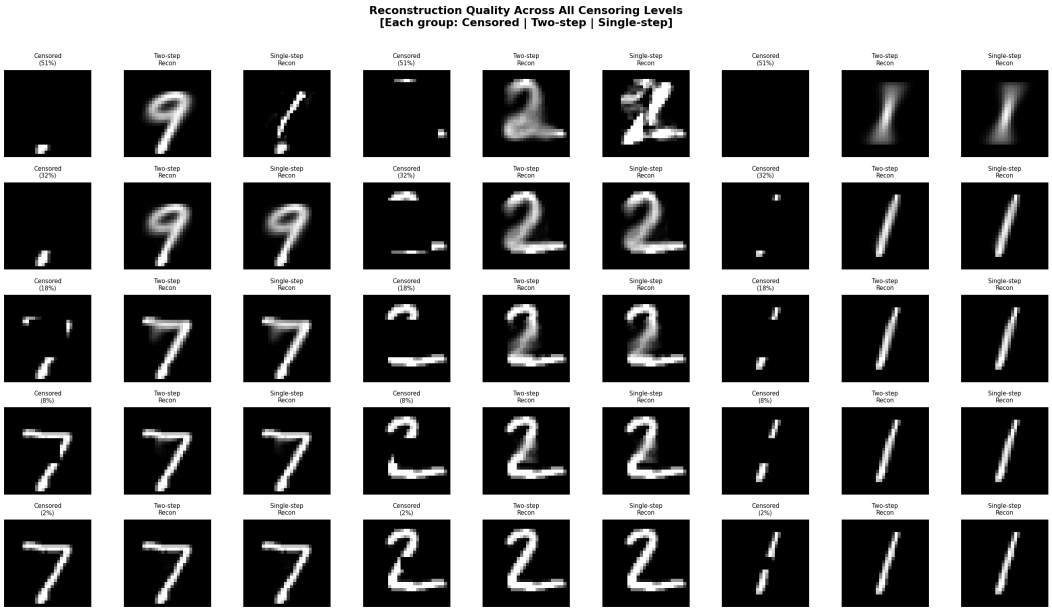


Figure 2: Reconstruction quality across all five censoring levels. Each group of three columns shows: the censored image, the two-step reconstruction, and the single-step reconstruction.